

Detailed Answer Sheet

.NET, C#, Dependency Injection, Collections, Delegates, LINQ, EF Core, ASP.NET Core API,
MVC/Razor, and Scenario-Based Questions

Prepared from the uploaded question list | Total questions answered: 220

Generated on: 12 May 2026

Contents

1. .NET and C# Fundamentals - 65 questions
2. Dependency Injection - 10 questions
3. Collections - 17 questions
4. Delegates and Events - 10 questions
5. LINQ - 23 questions
6. Entity Framework Core - 23 questions
7. ASP.NET Core / API - 30 questions
8. MVC / Razor - 22 questions
9. Advanced / Scenario-Based Questions - 20 questions

Note: Answers are written for interview, viva, and project explanation use. For modern platform questions, the wording assumes current .NET / ASP.NET Core practices unless legacy .NET Framework behavior is specifically mentioned.

Section 1: .NET and C# Fundamentals

Q1. What is the .NET ecosystem?

The .NET ecosystem is the complete platform around Microsoft .NET for building applications. It includes the .NET runtime, SDK, C# and other languages, base class libraries, ASP.NET Core, Entity Framework Core, desktop frameworks, cloud tooling, NuGet packages, Visual Studio/VS Code, and deployment tools. In simple words, it is the full environment used to develop, run, test, package, and deploy .NET applications.

Q2. What are the components of .NET?

Main components are: CLR/CoreCLR runtime, Base Class Library (BCL), Framework Class Library APIs, .NET SDK, compilers such as Roslyn for C#, Common Type System (CTS), Common Language Specification (CLS), garbage collector, JIT compiler, NuGet package manager, and application frameworks such as ASP.NET Core, EF Core, MAUI, WinForms, and WPF. These pieces together allow code to be written, compiled, executed, and deployed.

Q3. Difference between .NET Framework, .NET Core, and .NET?

.NET Framework is the older Windows-only platform mainly used for legacy desktop and ASP.NET Framework applications. .NET Core was the cross-platform, open-source redesign used for modern apps from version 1.0 to 3.1. From .NET 5 onward, the platform is simply called .NET; it continues the .NET Core line and is cross-platform, open-source, high performance, and recommended for new development.

Q4. What is CLR?

CLR stands for Common Language Runtime. It is the execution engine of .NET that runs managed code. It provides services such as memory management, garbage collection, exception handling, type safety, security checks, thread management, and Just-In-Time compilation from Intermediate Language to native machine code.

Q5. What are the responsibilities of CLR?

CLR loads assemblies, verifies type safety, converts IL into native code through JIT, manages memory through garbage collection, handles exceptions, manages threads, enforces security boundaries, supports interoperability with unmanaged code, and provides runtime services such as reflection and debugging support. Because CLR handles these services, developers can focus more on business logic.

Q6. What is CTS?

CTS stands for Common Type System. It defines how types are declared, used, and managed in .NET. It ensures that data types from different .NET languages are compatible. For example, C# int and VB.NET Integer both map to System.Int32, so code written in different .NET languages can work together.

Q7. What is CLS?

CLS stands for Common Language Specification. It is a set of rules that .NET languages should follow so that code written in one language can be used by another CLS-compliant language. It is a subset of CTS. For example, if a public library follows CLS rules, it can be safely consumed by C#, VB.NET, F#, and other compatible languages.

Q8. Difference between CTS and CLS?

CTS defines the complete type system of .NET, including all supported types and rules for how they behave. CLS defines a smaller set of rules that languages should follow for cross-language compatibility. CTS is broader; CLS is a compatibility guideline. Example: CTS may support unsigned types, but CLS may restrict them for public APIs because not every .NET language supports them.

Q9. How does .NET support multiple languages?

.NET supports multiple languages because compilers for C#, VB.NET, F#, and other .NET languages compile source code into a common Intermediate Language (IL). The IL follows CTS rules and is executed by CLR. Since all languages target the same runtime and type system, code written in one .NET language can call code written in another language.

Q10. What is managed code?

Managed code is code that runs under the control of CLR. CLR manages memory, garbage collection, type safety, exception handling, and security for this code. C# code compiled for .NET is usually managed code. Managed code is safer and easier to maintain because many low-level tasks are handled by the runtime.

Q11. What is unmanaged code?

Unmanaged code is code that runs directly on the operating system without CLR managing it. Examples include C or C++ native code, Windows API calls, and COM components. In unmanaged code, the developer is responsible for memory allocation, deallocation, and many safety concerns.

Q12. Difference between managed and unmanaged code?

Managed code runs inside CLR and receives services like garbage collection, type checking, and exception handling. Unmanaged code runs outside CLR and interacts directly with the operating system or hardware. Managed code is generally safer and easier to debug, while unmanaged code may be faster for low-level operations but requires manual memory and resource management.

Q13. What is interoperability in .NET?

Interoperability means the ability of .NET code to communicate with non-.NET code or systems. Examples include calling native DLLs using P/Invoke, using COM components, communicating with REST APIs, accessing databases, or integrating with legacy systems. It is useful when modern .NET applications need to reuse existing native or external components.

Q14. What is JIT compiler?

JIT stands for Just-In-Time compiler. It converts Intermediate Language (IL) into native machine code at runtime, just before execution. JIT compilation allows .NET applications to be portable because the same IL can run on different platforms, while the runtime generates machine-specific code for the current operating system and CPU.

Q15. What is IL (Intermediate Language)?

IL, also called MSIL or CIL, is the CPU-independent code generated after compiling a .NET language like C#. Source code is first compiled into IL and stored inside assemblies. At runtime, CLR uses the JIT compiler to convert IL into native machine code. IL makes cross-language and cross-platform execution possible.

Q16. What is assembly?

An assembly is the basic unit of deployment and versioning in .NET. It is usually a .dll or .exe file containing compiled IL code, metadata, resources, and manifest information. Assemblies describe the types they contain, their dependencies, version, culture, and security information.

Q17. Types of assemblies?

Common types are private assemblies and shared assemblies. A private assembly is used by a single application and is stored in that application's folder. A shared assembly can be used by multiple applications and, in .NET Framework, can be installed in the Global Assembly Cache. Assemblies can also be static or dynamic, and executable or library assemblies.

Q18. What is GAC?

GAC stands for Global Assembly Cache. It is a machine-wide storage location used mainly in .NET Framework for shared strongly named assemblies. Assemblies placed in the GAC can be used by multiple applications. In modern .NET/.NET Core, application-local deployment and NuGet package references are preferred instead of relying on the GAC.

Q19. What is garbage collection?

Garbage collection is automatic memory management in .NET. The garbage collector identifies objects on the managed heap that are no longer reachable by the application and frees their memory. This reduces memory leaks and removes the need for developers to manually delete most objects. However, unmanaged resources should still be released using `IDisposable` or using statements.

Q20. Generations in Garbage Collector?

The .NET garbage collector uses generations to improve performance. Generation 0 stores short-lived objects and is collected most frequently. Generation 1 acts as a buffer for objects that survived Gen 0. Generation 2 contains long-lived objects and is collected less often. Large objects are stored on the Large Object Heap and are managed separately.

Q21. What are nullable types?

Nullable value types allow value types such as `int`, `bool`, `DateTime`, and `decimal` to store null. They are written as `Nullable<T>` or using the shorthand `T?`, for example `int?`. They are useful when a value may be unknown or optional, such as a database column that allows `NULL`.

Q22. Difference between value type and reference type?

Value types store the actual data and are usually allocated on the stack or inline inside objects. Examples are `int`, `bool`, `decimal`, `struct`, and `enum`. Reference types store a reference to an object on the managed heap. Examples are `class`, `string`, `array`, `object`, and `interface`. Assigning a value type copies the value; assigning a reference type copies the reference.

Q23. What is boxing?

Boxing is the process of converting a value type into an object or interface reference. The value is copied from its value-type storage into an object on the heap. Example: `int x = 10; object obj = x;` Here, `x` is boxed into `obj`.

Q24. What is unboxing?

Unboxing is the reverse of boxing. It extracts the value type from an object reference back to its original value type. Example: `object obj = 10; int x = (int)obj;` The cast must match the original boxed type, otherwise an `InvalidCastException` can occur.

Q25. Performance impact of boxing/unboxing?

Boxing and unboxing can reduce performance because boxing allocates memory on the heap and copies the value, while unboxing requires type checking and copying back. Frequent boxing can increase garbage collection pressure. Generic collections such as `List<int>` avoid boxing, which is one major reason they are preferred over non-generic collections like `ArrayList`.

Q26. What is namespace?

A namespace is a logical container used to organize related classes, interfaces, enums, structs, and other types. It prevents naming conflicts and improves code readability. Example: `System.Collections.Generic` contains collection-related types. In a project, namespaces usually follow the folder or application structure.

Q27. What is class?

A class is a blueprint for creating objects. It defines data through fields/properties and behavior through methods. For example, a `Customer` class can contain `Name`, `Email`, and `PlaceOrder()`. A class supports object-oriented concepts such as encapsulation, inheritance, polymorphism, and abstraction.

Q28. Difference between class and object?

A class is a blueprint or template; an object is a real instance created from that class. For example, `Car` is a class, while `myCar` is an object of the `Car` class. The class defines what data and behavior exist, and the object stores actual values at runtime.

Q29. What is object?

An object is an instance of a class created at runtime. It has state and behavior. State is represented by fields or properties, and behavior is represented by methods. In C#, object is also the base type `System.Object` from which all types ultimately derive.

Q30. What are access specifiers?

Access specifiers define where a class, method, property, field, or constructor can be accessed from. They help enforce encapsulation and protect internal implementation details. Examples include `public`, `private`, `protected`, `internal`, `protected internal`, `private protected`, and `file` in newer C# versions.

Q31. Difference between access specifier and modifier?

An access specifier controls accessibility, for example `public` or `private`. A modifier changes behavior or meaning, for example `static`, `abstract`, `sealed`, `virtual`, `override`, `readonly`, `async`, or `partial`. All access specifiers are modifiers, but not all modifiers are access specifiers.

Q32. Types of access modifiers in C#?

Common access modifiers are public, private, protected, internal, protected internal, private protected, and file. public is accessible everywhere, private only inside the same type, protected inside the same type and derived types, internal inside the same assembly, and combined forms provide mixed assembly and inheritance-based access.

Q33. What is constructor?

A constructor is a special method used to initialize an object when it is created. It has the same name as the class and does not have a return type. Constructors can initialize fields, assign default values, validate input, or inject dependencies in classes such as services and controllers.

Q34. Types of constructors?

Common constructor types are default constructor, parameterized constructor, static constructor, private constructor, and copy constructor pattern. A default constructor has no parameters. A parameterized constructor accepts values. A static constructor initializes static members. A private constructor prevents direct object creation from outside the class.

Q35. What happens if constructor is not present?

If no constructor is defined in a class, C# automatically provides a public parameterless default constructor, provided the class itself is accessible. This constructor initializes fields to their default values, such as 0 for numbers, false for bool, and null for reference types.

Q36. What is default constructor?

A default constructor is a constructor that does not take parameters. It initializes an object with default values. If the developer does not define any constructor, the compiler provides one automatically. If any constructor is defined manually, the compiler does not automatically generate the parameterless one.

Q37. What is static keyword?

static means the member belongs to the type itself rather than to an individual object. A static field, method, property, or constructor is accessed using the class name. Static members are useful for shared data, utility methods, constants, factory helpers, and application-wide logic that does not depend on object state.

Q38. Where do we use static?

Use static for utility/helper methods, constants, shared configuration, extension methods, factory methods, counters shared by all objects, and application entry points such as Main. Avoid using static for business services that need dependency injection or unit testing, because static members can make code tightly coupled.

Q39. Difference between static and instance members?

Static members belong to the class and are shared by all objects. Instance members belong to a specific object and can hold different values for each object. Static methods cannot directly access instance members unless they use an object reference, while instance methods can access both instance members and static members.

Q40. Can constructor be static?

Yes, a class can have a static constructor. It is used to initialize static members and runs automatically once before the class is first used. A static constructor has no access modifier, no parameters, and cannot be called manually. It is different from an instance constructor, which runs whenever an object is created.

Q41. What is inheritance?

Inheritance is an object-oriented concept where one class derives from another class and reuses or extends its members. The parent class is called base class, and the child class is called derived class. It promotes code reuse and allows polymorphism through virtual and overridden members.

Q42. Types of inheritance in C#?

C# supports single inheritance with classes, multilevel inheritance, hierarchical inheritance, and multiple inheritance through interfaces. It does not support multiple inheritance with classes. A class can inherit only one class but can implement multiple interfaces.

Q43. Why multiple inheritance is not supported with classes?

Multiple inheritance with classes can create ambiguity, especially when two base classes contain members with the same name or shared inheritance paths. This is known as the diamond problem. C# avoids this complexity by allowing only one base class but supports multiple interfaces, which provide safer abstraction contracts.

Q44. What is polymorphism?

Polymorphism means one interface or method call can behave differently depending on the actual object or method signature. It allows flexible and extensible code. In C#, compile-time polymorphism is achieved using method overloading, and runtime polymorphism is achieved using method overriding with virtual, override, and abstract members.

Q45. Compile-time vs runtime polymorphism?

Compile-time polymorphism is resolved by the compiler, usually through method overloading or operator overloading. Runtime polymorphism is resolved at runtime based on the actual object type, usually through virtual methods, abstract methods, or interface implementation. Runtime polymorphism is heavily used in dependency injection and service abstractions.

Q46. What is abstraction?

Abstraction means hiding implementation details and showing only essential behavior. It helps users of a class focus on what an object does, not how it does it. In C#, abstraction is achieved using abstract classes, interfaces, access modifiers, and clean service contracts.

Q47. What is encapsulation?

Encapsulation means wrapping data and methods inside a class and controlling access to that data. It protects object state from invalid changes. In C#, encapsulation is achieved using private fields, public properties, validation logic, and access modifiers.

Q48. What is abstract class?

An abstract class is a class that cannot be directly instantiated. It can contain abstract members without implementation and concrete members with implementation. It is used when related classes share common behavior but some behavior must be implemented differently by derived classes.

Q49. What is interface?

An interface defines a contract that a class or struct can implement. It mainly contains method, property, event, or indexer declarations. Modern C# also supports default interface implementations, but interfaces are still mainly used to define behavior without tightly coupling to a concrete class.

Q50. Difference between abstract class and interface?

An abstract class is used for a base class relationship and can contain fields, constructors, concrete methods, and abstract methods. An interface defines a contract and supports multiple implementation. A class can inherit one abstract class but implement multiple interfaces. Use abstract class for shared base behavior; use interface for capability and loose coupling.

Q51. When should you use abstract class?

Use an abstract class when classes are closely related and share common state or behavior. It is suitable when you need protected members, constructors, fields, or partial implementation. Example: a base `PaymentProcessor` class can contain common validation and logging, while derived classes implement specific payment logic.

Q52. When should you use interface?

Use an interface when you want to define what a class can do without forcing a shared base class. Interfaces are ideal for dependency injection, unit testing, plugin-like design, and loosely coupled architecture. Example: `ICustomerRepository` allows services to depend on an abstraction instead of a specific repository class.

Q53. Can abstract class have constructor?

Yes, an abstract class can have a constructor. Even though the abstract class cannot be instantiated directly, its constructor runs when a derived class object is created. It is used to initialize common state required by derived classes.

Q54. Can interface contain implementation?

Traditionally interfaces contained only declarations. In modern C#, interfaces can contain default implementations for members. However, interfaces should still be used carefully for contracts; putting too much implementation inside an interface can reduce clarity and make design harder to maintain.

Q55. What is method overriding?

Method overriding occurs when a derived class provides a new implementation for a base class virtual or abstract method using the override keyword. It enables runtime polymorphism. The method signature must remain the same, and the call is resolved based on the actual object type at runtime.

Q56. What is method overloading?

Method overloading means defining multiple methods with the same name but different parameter lists in the same class. The difference can be number, type, or order of parameters. It is compile-time polymorphism. Return type alone is not enough to overload a method.

Q57. What is method hiding?

Method hiding occurs when a derived class defines a member with the same name as a base class member using the new keyword. Unlike overriding, hiding does not provide runtime polymorphism. The method called depends on the reference type, not the actual object type.

Q58. Difference between overriding and hiding?

Overriding uses virtual/abstract in the base class and override in the derived class. It supports runtime polymorphism. Hiding uses new in the derived class and does not replace the base implementation polymorphically. With overriding, the actual object type decides the method; with hiding, the reference type decides.

Q59. What is sealed class?

A sealed class is a class that cannot be inherited. It is declared using the sealed keyword. It is used when a class should not be extended for design, security, or performance reasons. Example: string is sealed. A method can also be sealed to stop further overriding in derived classes.

Q60. What is partial class?

A partial class allows a class definition to be split across multiple files using the partial keyword. At compile time, all parts are combined into one class. It is useful in generated code, designer files, and large classes where separating generated and manual code improves maintainability.

Q61. What is extension method?

An extension method adds new methods to an existing type without modifying the original type. It is a static method inside a static class, and its first parameter uses the this keyword. LINQ methods such as Where and Select are common examples of extension methods.

Q62. What is object initializer?

An object initializer allows setting property values while creating an object without explicitly calling multiple assignment statements. Example: new Customer { Name = "Amit", Email = "a@x.com" }. It improves readability when constructing objects with public settable properties.

Q63. What is anonymous type?

An anonymous type is a type created by the compiler without explicitly defining a class. It is commonly used in LINQ projections. Example: var item = new { Name = "Amit", Age = 22 }; Anonymous types are read-only and mainly used for temporary data shapes.

Q64. What is nullable reference type?

Nullable reference types are a C# feature that helps detect possible null reference errors at compile time. When enabled, string means a non-null string is expected, while string? means null is allowed. It does not change runtime behavior directly; it adds compiler warnings to improve null-safety.

Q65. Difference between == and .Equals()?

== is an operator that may compare references or values depending on the type and operator overloads. .Equals() is a method used to check equality and can be overridden by types. For reference types, == often checks reference equality unless overloaded, while Equals usually represents logical equality when properly overridden.

Section 2: Dependency Injection

Q66. What is Dependency Injection?

Dependency Injection (DI) is a design pattern where a class receives its required dependencies from outside instead of creating them itself. For example, a controller receives an `ICustomerService` through its constructor. This makes the code loosely coupled, easier to test, and easier to change.

Q67. Why DI is needed?

DI is needed to reduce tight coupling between classes. Without DI, a class creates its own dependencies using `new`, making replacement, unit testing, and maintenance difficult. With DI, the application can provide different implementations without changing the dependent class.

Q68. Benefits of DI?

Benefits include loose coupling, easier unit testing, better maintainability, improved separation of concerns, centralized object creation, easier replacement of implementations, and cleaner architecture. In ASP.NET Core, DI also helps manage object lifetimes such as Singleton, Scoped, and Transient.

Q69. Types of Dependency Injection?

Common types are constructor injection, property injection, and method injection. Constructor injection is most preferred because it clearly shows required dependencies and ensures the object is created in a valid state. Property and method injection are used for optional or action-specific dependencies.

Q70. What is constructor injection?

Constructor injection means dependencies are passed through a class constructor. Example: a controller receives `IProductService` in its constructor. This is the recommended approach in ASP.NET Core because required dependencies are visible, immutable, and available as soon as the object is created.

Q71. What is service lifetime?

Service lifetime defines how long an object created by the DI container lives. ASP.NET Core mainly provides three lifetimes: Singleton, Scoped, and Transient. Correct lifetime selection is important for memory usage, performance, database context handling, and avoiding shared-state bugs.

Q72. Difference between Singleton, Scoped, and Transient?

Singleton creates one instance for the whole application lifetime. Scoped creates one instance per request, which is commonly used for `DbContext` and business services in web apps. Transient creates a new instance every time it is requested. Singleton is shared, Scoped is per request, and Transient is per injection.

Q73. What problems does DI solve?

DI solves tight coupling, hard-coded dependencies, difficult unit testing, repeated object creation, and poor separation of concerns. It also allows centralized configuration of services and lifetimes. In layered architecture, DI helps controllers depend on services, and services depend on repositories through interfaces.

Q74. What is IOC Container?

IOC stands for Inversion of Control. An IOC container is responsible for creating objects, resolving dependencies, and managing lifetimes. In ASP.NET Core, the built-in service container is configured through `builder.Services` and resolves dependencies automatically at runtime.

Q75. Explain DI in ASP.NET Core.

ASP.NET Core has built-in dependency injection. Services are registered in `Program.cs` using methods such as `AddScoped`, `AddTransient`, and `AddSingleton`. The framework then injects those services into controllers, middleware, filters, repositories, or other services through constructors. Example flow: Controller asks for `ICustomerService`, DI container creates `CustomerService`, and also injects its repository dependencies.

Section 3: Collections

Q76. What is collection?

A collection is an object used to store, manage, and access a group of related items. Collections provide operations such as add, remove, search, sort, and iterate. In C#, collections are available in namespaces such as `System.Collections` and `System.Collections.Generic`.

Q77. Difference between array and collection?

An array has a fixed size after creation and stores elements of the same type. A collection is usually dynamic and can grow or shrink at runtime. Collections also provide more built-in operations such as `Add`, `Remove`, `Contains`, sorting, key-value lookup, and LINQ support.

Q78. What are non-generic collections?

Non-generic collections store elements as object, so they can hold different types. Examples include `ArrayList`, `Hashtable`, `Stack`, and `Queue` from `System.Collections`. They are less type-safe and can require boxing/unboxing for value types.

Q79. Examples of non-generic collections?

Common non-generic collections are `ArrayList`, `Hashtable`, `SortedList`, `Stack`, `Queue`, and `BitArray`. These were used heavily before generics were introduced. In modern C#, generic collections are preferred because they provide compile-time type safety and better performance.

Q80. What are generic collections?

Generic collections are strongly typed collections where the element type is specified at compile time. Examples include `List<T>`, `Dictionary<TKey,TValue>`, `Queue<T>`, `Stack<T>`, `HashSet<T>`, and `SortedDictionary<TKey,TValue>`. They avoid unnecessary casting and reduce runtime errors.

Q81. Why generic collections are preferred?

Generic collections are preferred because they provide type safety, better performance, no boxing/unboxing for value types, cleaner code, and compile-time error detection. Example: `List<int>` can store only integers, while `ArrayList` can accidentally store any object.

Q82. Difference between generic and non-generic collections?

Generic collections are strongly typed and use type parameters such as `List<int>`. Non-generic collections store items as object. Generic collections reduce casting and runtime errors, and they improve performance by avoiding boxing for value types. Non-generic collections are mostly used only in legacy code.

Q83. What is ArrayList?

`ArrayList` is a non-generic dynamic collection that stores elements as object. It can grow automatically, but it is not type-safe. Since it stores value types as object, boxing and unboxing may occur. In modern C#, `List<T>` is preferred over `ArrayList`.

Q84. What is Hashtable?

`Hashtable` is a non-generic key-value collection. It stores keys and values as object and uses hashing for fast lookup. It is not type-safe, so wrong key or value types can cause runtime errors. In modern C#, `Dictionary<TKey,TValue>` is preferred.

Q85. What is List<T>?

`List<T>` is a generic dynamic array collection. It stores elements of a specific type and can grow automatically. It supports indexing, `Add`, `Remove`, `Contains`, sorting, and LINQ operations. It is one of the most commonly used collections in C#.

Q86. What is Dictionary<TKey,TValue>?

`Dictionary<TKey,TValue>` is a generic key-value collection. It provides fast lookup using keys. Each key must be unique, and each key maps to a value. Example: `Dictionary<int, string>` can store student IDs as keys and names as values.

Q87. Difference between List and Array?

An array has fixed size and lower overhead, while `List<T>` is dynamic and provides many helper methods. Arrays are useful when size is fixed and performance is critical. `List<T>` is preferred when items need to be added or removed frequently.

Q88. What is Queue?

A Queue is a First-In, First-Out (FIFO) collection. The item added first is removed first. It is useful for task processing, print queues, request handling, and background job processing. In C#, `Queue<T>` is the generic version.

Q89. What is Stack?

A Stack is a Last-In, First-Out (LIFO) collection. The last item added is removed first. It is useful for undo operations, expression evaluation, recursion simulation, browser history, and backtracking algorithms. In C#, `Stack<T>` is the generic version.

Q90. What is HashSet?

`HashSet<T>` is a generic collection that stores unique elements only. It is useful when duplicates are not allowed and fast lookup is needed. It supports set operations such as `UnionWith`, `IntersectWith`, and `ExceptWith`.

Q91. What is jagged array?

A jagged array is an array of arrays. Each inner array can have a different length. Example: `int[][] data = new int[3][];` Here each row can contain a different number of columns. It is useful for irregular data structures.

Q92. Difference between jagged array and multidimensional array?

A jagged array is an array of arrays, and each row can have a different length. A multidimensional array is rectangular and has fixed dimensions, such as `int[,]` matrix. Jagged arrays can be more flexible, while multidimensional arrays are better for true matrix-like data.

Section 4: Delegates and Events

Q93. What is delegate?

A delegate is a type-safe reference to a method. It defines a method signature and can point to any method with the same signature. Delegates are used to pass methods as parameters, implement callbacks, and support events and LINQ expressions.

Q94. Why delegates are used?

Delegates are used when a method needs to call another method dynamically or when behavior should be passed as data. They are useful for callbacks, event handling, strategy selection, plug-in style design, and LINQ methods such as Where, Select, and OrderBy.

Q95. Types of delegates?

Common delegate types include single-cast delegates, multicast delegates, generic delegates, anonymous delegates, and built-in delegates such as Action, Func, and Predicate. Single-cast points to one method; multicast can point to multiple methods.

Q96. What is generic delegate?

A generic delegate uses type parameters so it can work with different data types. Built-in examples are Action<T>, Func<T, TResult>, and Predicate<T>. Generic delegates reduce the need to create custom delegate types for common method signatures.

Q97. Difference between Action, Func, and Predicate?

Action represents a method that returns void. Func represents a method that returns a value, with the last type parameter as the return type. Predicate represents a method that takes one input and returns bool. Example: Predicate<int> can check whether a number is even.

Q98. What is multicast delegate?

A multicast delegate holds references to multiple methods. When invoked, it calls all methods in its invocation list in order. It is commonly used in event handling. If multicast delegates return values, only the result of the last method is returned, so they are usually used with void methods.

Q99. What is anonymous method?

An anonymous method is a method without a name, written inline using the delegate keyword. It allows passing behavior without declaring a separate named method. Lambda expressions are now more commonly used because they are shorter and more readable.

Q100. What is lambda expression?

A lambda expression is a concise way to write an anonymous function. Example: `x => x > 10`. Lambdas are widely used in LINQ, delegates, event handling, and expression trees. They make code shorter and allow behavior to be passed as a parameter.

Q101. What is event?

An event is a notification mechanism built on delegates. It allows one class to notify other classes when something happens, such as a button click or data change. Events provide controlled access so subscribers can add or remove handlers but cannot directly invoke the event from outside the declaring class.

Q102. Difference between delegate and event?

A delegate is a type that can reference methods and can be invoked directly if accessible. An event is a wrapper around a delegate that restricts outside classes to only subscribe or unsubscribe. Events are safer for publisher-subscriber scenarios because only the declaring class can raise the event.

Section 5: LINQ

Q103. What is LINQ?

LINQ stands for Language Integrated Query. It allows querying collections, databases, XML, and other data sources using C# syntax. LINQ provides a consistent way to filter, sort, group, join, and transform data.

Q104. Why LINQ is used?

LINQ is used because it makes data querying readable, strongly typed, and integrated with C#. It reduces manual loops, improves maintainability, supports compile-time checking, and works with many data sources such as List<T>, arrays, EF Core queries, and XML.

Q105. Difference between method syntax and query syntax?

Method syntax uses extension methods and lambda expressions, for example `students.Where(s => s.Age > 18)`. Query syntax looks like SQL, for example `from s in students where s.Age > 18 select s`. Both are converted internally to method calls. Method syntax supports all LINQ operators; query syntax is often more readable for complex joins.

Q106. What is deferred execution?

Deferred execution means a LINQ query is not executed when it is defined. It is executed only when the result is enumerated, such as in a `foreach` loop or when `ToList()` is called. This allows queries to be built step by step and executed only when needed.

Q107. What is immediate execution?

Immediate execution means the query runs immediately and returns a concrete result. Methods such as `ToList()`, `ToArray()`, `Count()`, `First()`, `Single()`, `Any()`, `Max()`, `Min()`, and `Sum()` trigger immediate execution. Immediate execution is useful when the result must be materialized or evaluated immediately.

Q108. What methods cause immediate execution?

Common immediate execution methods are `ToList()`, `ToArray()`, `ToDictionary()`, `Count()`, `LongCount()`, `First()`, `FirstOrDefault()`, `Single()`, `SingleOrDefault()`, `Any()`, `All()`, `Max()`, `Min()`, `Sum()`, `Average()`, and `Aggregate()`. These methods force LINQ to process the query immediately.

Q109. What is default execution behavior in LINQ?

Most LINQ operators such as `Where`, `Select`, `OrderBy`, `GroupBy`, `Skip`, and `Take` use deferred execution. They build a query pipeline and execute only when enumerated. However, terminal operators such as `ToList`, `Count`, `First`, and `Any` execute immediately.

Q110. What is Select?

`Select` is a projection operator. It transforms each item in a sequence into a new form. Example: `customers.Select(c => c.Name)` returns only customer names. In EF Core, `Select` is commonly used to shape data into DTOs and avoid returning unnecessary columns.

Q111. Difference between Select and SelectMany?

`Select` transforms each source item into one result. `SelectMany` flattens nested collections into a single sequence. Example: if each customer has many orders, `Select` returns a sequence of order collections, while `SelectMany` returns one flat sequence of all orders.

Q112. What is Where?

`Where` is a filtering operator. It returns only elements that satisfy a condition. Example: `products.Where(p => p.Price > 1000)`. In EF Core, `Where` is translated into SQL `WHERE` conditions when used with `IQueryable`.

Q113. What is OrderBy?

`OrderBy` sorts a sequence in ascending order based on a key. Example: `students.OrderBy(s => s.Name)`. For descending order, use `OrderByDescending`. In databases, EF Core translates it into `ORDER BY SQL`.

Q114. Difference between OrderBy and ThenBy?

OrderBy starts a new primary sort. ThenBy adds a secondary sort after an existing OrderBy. Example: `OrderBy(s => s.City).ThenBy(s => s.Name)` sorts first by city and then by name within each city. Calling OrderBy again replaces the previous sort priority.

Q115. What is GroupBy?

GroupBy groups elements based on a key. Example: `employees.GroupBy(e => e.DepartmentId)` creates groups of employees by department. It is useful for summaries, counts, totals, and categorization.

Q116. What is Join?

Join combines two sequences based on matching keys. It is similar to SQL INNER JOIN. Example: joining orders with customers using `CustomerId`. In LINQ, Join is useful when related data is stored in separate collections or tables.

Q117. Types of joins in LINQ?

Common joins include inner join, group join, left outer join, cross join, and self join. LINQ does not have a direct left join keyword; it is usually implemented using `GroupJoin` with `DefaultIfEmpty()`. EF Core can translate many join patterns into SQL joins.

Q118. Difference between First and FirstOrDefault?

First returns the first element that matches a condition and throws an exception if no element exists. `FirstOrDefault` returns the first matching element or the default value if none exists, such as null for reference types or 0 for int.

Q119. Difference between Single and SingleOrDefault?

Single expects exactly one matching element and throws an exception if none or more than one exists. `SingleOrDefault` returns the single matching element or default if none exists, but still throws if more than one matching element exists. Use Single when uniqueness is required.

Q120. What is Any()?

Any() checks whether a sequence contains at least one element. With a predicate, it checks whether at least one element satisfies the condition. It returns a bool. In database queries, Any() is efficient because it can stop as soon as a match is found.

Q121. Difference between Any() and Count()?

Any() checks existence and can stop after finding the first matching element. Count() counts all matching elements. For checking whether records exist, Any() is usually more efficient than `Count() > 0`, especially with databases.

Q122. What is Skip and Take?

Skip ignores a specified number of elements, and Take returns a specified number of elements. They are commonly used for pagination. Example: `query.Skip((pageNumber - 1) * pageSize).Take(pageSize)`. In EF Core, this becomes OFFSET and FETCH in SQL Server.

Q123. What is Distinct?

Distinct removes duplicate elements from a sequence. For primitive values, it compares values directly. For custom objects, equality depends on Equals/GetHashCode or a custom comparer. In database queries, it translates to SQL DISTINCT.

Q124. What is Union?

Union combines two sequences and removes duplicates. It returns unique elements from both sequences. For example, `list1.Union(list2)` gives all unique values present in either list. In SQL translation, it behaves like UNION rather than UNION ALL.

Q125. Difference between IEnumerable and IQueryable?

IEnumerable is used for in-memory iteration and LINQ to Objects. IQueryable represents a query that can be translated and executed by a provider such as EF Core. With IQueryable, filtering can happen in the database; with IEnumerable, data may already be loaded into memory before filtering.

Section 6: Entity Framework Core

Q126. What is Entity Framework Core?

Entity Framework Core is a modern object-relational mapper (ORM) for .NET. It allows developers to work with databases using C# classes instead of writing SQL for every operation. EF Core maps classes to tables, properties to columns, and relationships to foreign keys.

Q127. Why EF Core is used?

EF Core is used to simplify database access, reduce repetitive SQL code, support LINQ queries, track changes, handle relationships, and manage migrations. It improves productivity and allows developers to work with strongly typed C# models while still using relational databases like SQL Server, PostgreSQL, MySQL, or SQLite.

Q128. Difference between ADO.NET and EF Core?

ADO.NET is a lower-level data access technology where developers manually write SQL, open connections, execute commands, and map results. EF Core is a higher-level ORM that handles mapping, tracking, relationships, and LINQ translation. ADO.NET gives more direct control; EF Core gives faster development and cleaner domain code.

Q129. What is DbContext?

DbContext is the main EF Core class that represents a session with the database. It manages database connections, entity tracking, LINQ query execution, SaveChanges, transactions, and model configuration. In ASP.NET Core, DbContext is usually registered as a scoped service.

Q130. What is DbSet?

DbSet<TEntity> represents a table or collection of entities in the database. It is used to query, add, update, and remove records for a specific entity type. Example: DbSet<Customer> Customers represents the Customers table.

Q131. What is migration?

A migration is a file that records changes made to the EF Core model so the database schema can be updated. It contains Up and Down methods. Up applies schema changes, while Down rolls them back. Migrations are mainly used in Code First development.

Q132. Why migration class is required in Code First?

In Code First, the C# model is treated as the source of truth. A migration class captures the difference between the current model and the previous database schema. Without migrations, EF Core would not know how to safely create, update, or roll back database schema changes.

Q133. Difference between Code First, Database First, and Model First?

Code First starts with C# classes and creates/updates the database using migrations. Database First starts with an existing database and scaffolds C# models and DbContext from it. Model First starts with a visual/design model and generates database/code; it was more common in older EF, not EF Core modern workflow.

Q134. What is change tracking?

Change tracking is EF Core's mechanism for monitoring entity objects loaded by DbContext. It detects added, modified, deleted, and unchanged entities. When SaveChanges() is called, EF Core generates SQL commands based on these tracked changes.

Q135. What is eager loading?

Eager loading loads related data as part of the initial query using Include and ThenInclude. Example: context.Orders.Include(o => o.Customer). It is useful when you know related data is needed immediately and want to avoid multiple database calls.

Q136. What is lazy loading?

Lazy loading loads related data only when the navigation property is accessed. It can make code simple but may cause many hidden database queries, known as the N+1 problem. EF Core supports lazy loading through proxies or ILazyLoader configuration.

Q137. What is explicit loading?

Explicit loading loads related data manually after the main entity is already loaded. It uses `Entry().Reference().Load()` or `Entry().Collection().Load()`. It gives control over exactly when related data is loaded without loading everything upfront.

Q138. Difference between eager, lazy, and explicit loading?

Eager loading loads related data with the initial query. Lazy loading loads related data automatically when accessed. Explicit loading loads related data manually when requested. Eager is predictable, lazy is convenient but can create extra queries, and explicit gives manual control.

Q139. What is navigation property?

A navigation property represents a relationship between entities. For example, an Order can have a Customer navigation property, and a Customer can have a collection of Orders. EF Core uses navigation properties with foreign keys to understand relationships.

Q140. What is primary key?

A primary key uniquely identifies each row in a table. It cannot be null and should be unique. In EF Core, a property named `Id` or `EntityNameId` is often automatically recognized as the primary key by convention.

Q141. What is foreign key?

A foreign key is a column that links one table to the primary key of another table. It represents a relationship between tables. Example: `Order.CustomerId` is a foreign key referencing `Customer.CustomerId`. EF Core can configure foreign keys using conventions, data annotations, or Fluent API.

Q142. What is Fluent API?

Fluent API is a way to configure EF Core models using code in `OnModelCreating`. It can define keys, relationships, column names, data types, constraints, indexes, table names, and delete behavior. It is more powerful than data annotations.

Q143. What are data annotations?

Data annotations are attributes applied to model properties or classes to configure validation and mapping. Examples include `[Key]`, `[Required]`, `[MaxLength]`, `[Column]`, and `[ForeignKey]`. They are simple and readable, but Fluent API is better for complex configurations.

Q144. What is LINQ to Entities?

LINQ to Entities is the use of LINQ queries against EF Core entity models. EF Core translates these LINQ expressions into SQL queries for the database. Not every C# method can be translated, so queries should use operations supported by the database provider.

Q145. What is AsNoTracking()?

`AsNoTracking()` tells EF Core not to track the returned entities in the `DbContext`. It improves performance for read-only queries because EF does not need to store tracking information. It should be used when data is only being displayed and not updated.

Q146. What is repository pattern?

Repository pattern is a design pattern that abstracts data access logic behind interfaces and classes. It allows services to call repository methods instead of directly writing EF Core queries everywhere. Example: `ICustomerRepository` can expose `GetAllAsync`, `GetByIdAsync`, `AddAsync`, and `DeleteAsync`.

Q147. What is Unit of Work?

Unit of Work coordinates multiple repository operations and commits them as a single transaction. In EF Core, `DbContext` already acts like a Unit of Work because it tracks changes across entities and saves them together using `SaveChanges` or `SaveChangesAsync`.

Q148. What is concurrency in EF Core?

Concurrency occurs when multiple users or processes try to update the same data at the same time. EF Core supports optimistic concurrency using concurrency tokens such as `rowversion`/`timestamp` columns. If data changes between read and update, EF Core can throw `DbUpdateConcurrencyException` so the conflict can be handled.

Section 7: ASP.NET Core / API

Q149. What is API?

API stands for Application Programming Interface. It is a set of rules and endpoints that allows one software system to communicate with another. In web development, an API usually exposes HTTP endpoints that clients such as web apps, mobile apps, or other services can call.

Q150. What is REST API?

A REST API is an API designed using REST principles. It exposes resources through URLs and uses HTTP methods such as GET, POST, PUT, PATCH, and DELETE. It is stateless and usually returns data in JSON format.

Q151. Principles of RESTful API?

RESTful APIs are resource-based, stateless, use standard HTTP methods, use proper status codes, support representation formats such as JSON, and have a uniform interface. Good REST APIs also use clear URLs, versioning when needed, and proper authentication/authorization.

Q152. Difference between REST and SOAP?

REST is an architectural style that commonly uses HTTP and JSON. It is lightweight, flexible, and widely used in modern web/mobile apps. SOAP is a protocol that uses XML messages and strict standards. SOAP is heavier but provides built-in standards for enterprise features such as formal contracts and WS-Security.

Q153. HTTP methods in API?

Common HTTP methods are GET for reading data, POST for creating data, PUT for full update, PATCH for partial update, DELETE for deletion, HEAD for headers only, and OPTIONS for supported operations or CORS preflight requests.

Q154. Difference between GET and POST?

GET is used to retrieve data and should not change server state. Parameters are often sent in the URL. POST is used to create data or perform operations that change state. POST sends data in the request body and is not idempotent by default.

Q155. Difference between PUT and PATCH?

PUT usually replaces or updates the complete resource. PATCH updates only selected fields of a resource. Example: PUT may send a full customer object, while PATCH may send only the email field to update.

Q156. Difference between PUT and POST?

POST is commonly used to create a new resource and lets the server decide the new resource ID. PUT is commonly used to update or replace a resource at a known URL. PUT is expected to be idempotent, meaning repeated identical requests should produce the same result.

Q157. What are HTTP status codes?

HTTP status codes indicate the result of an API request. 2xx means success, 3xx means redirection, 4xx means client error, and 5xx means server error. Examples: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, and 500 Internal Server Error.

Q158. Types of return types in API?

ASP.NET Core API actions can return specific types, IActionResult, ActionResult<T>, IActionResult in minimal APIs, or async Task versions of these. IActionResult is flexible for multiple response types, while ActionResult<T> combines strong typing with HTTP result support.

Q159. Difference between IActionResult and ActionResult<T>?

IActionResult is an interface that can return any HTTP result, such as Ok, NotFound, BadRequest, or Created. ActionResult<T> is strongly typed and documents the expected response model better. For modern APIs, ActionResult<T> is preferred when the endpoint usually returns a specific data type.

Q160. What is middleware?

Middleware is a component in the ASP.NET Core request pipeline. Each middleware can inspect, modify, pass, or stop an HTTP request/response. Examples include authentication, authorization, routing, exception handling, CORS, static files, and custom logging middleware.

Q161. What is request pipeline?

The request pipeline is the ordered sequence of middleware components that handle every HTTP request. A request passes through middleware in order, reaches the endpoint, and the response travels back through the pipeline. Middleware order is important because earlier middleware can affect later middleware.

Q162. Explain ASP.NET Core request pipeline.

When a request reaches an ASP.NET Core app, it enters the middleware pipeline configured in Program.cs. Common flow: exception handling, HTTPS redirection, static files, routing, CORS, authentication, authorization, endpoint execution, and response generation. Each middleware can call `next()` to continue or short-circuit the request.

Q163. What is dependency injection in ASP.NET Core?

Dependency injection in ASP.NET Core is the built-in mechanism for registering and resolving services. Services are registered in Program.cs using `builder.Services`. Controllers, services, repositories, middleware, and filters can receive dependencies through constructors. This supports clean layered architecture and testability.

Q164. What is routing?

Routing maps incoming HTTP requests to endpoints such as controller actions. It uses URL patterns, route parameters, HTTP methods, and attributes to decide which action should handle a request. Example: `GET /api/products/5` maps to a product controller action with `id = 5`.

Q165. Attribute routing vs conventional routing?

Attribute routing defines routes directly on controllers and actions using attributes like `[Route]` and `[HttpGet]`. Conventional routing defines route patterns globally, often in MVC apps. Attribute routing is preferred for Web APIs because endpoint URLs are explicit and easy to control.

Q166. What are filters in ASP.NET Core?

Filters are components that run before or after certain stages of action execution. They are used for cross-cutting concerns such as authorization, validation, exception handling, logging, caching, and response modification. Filters are especially common in MVC and controller-based APIs.

Q167. Types of filters?

Main filter types are authorization filters, resource filters, action filters, exception filters, and result filters. Authorization filters run first, resource filters wrap most of the pipeline, action filters run before/after action methods, exception filters handle exceptions, and result filters run around result execution.

Q168. What is authentication?

Authentication is the process of verifying who the user is. Examples include username/password login, JWT bearer token validation, cookies, OAuth, or external providers. If authentication succeeds, the application creates a user identity with claims.

Q169. What is authorization?

Authorization decides what an authenticated user is allowed to access. It checks roles, policies, claims, or permissions. Example: a user may be authenticated, but only an Admin role may access user management endpoints.

Q170. Difference between authentication and authorization?

Authentication verifies identity: who are you? Authorization verifies permission: what are you allowed to do? Authentication happens first, then authorization. Example: login proves the user identity; role-based checks decide whether they can access admin APIs.

Q171. What is JWT token?

JWT stands for JSON Web Token. It is a compact token format used to securely transmit claims between client and server. In APIs, after login the server issues a JWT, and the client sends it in the Authorization header as Bearer token. The server validates the token signature, issuer, audience, expiry, and claims.

Q172. What is CORS?

CORS stands for Cross-Origin Resource Sharing. It is a browser security mechanism that controls whether a web page from one origin can call an API on another origin. ASP.NET Core can configure allowed origins, methods, headers, and credentials using CORS policies.

Q173. What is model binding?

Model binding maps incoming request data to action method parameters or model objects. It can bind data from route values, query strings, headers, form data, and request body. Example: route id maps to int id, and JSON body maps to a DTO object.

Q174. What is model validation?

Model validation checks whether incoming data satisfies rules such as required fields, length, range, format, and custom validation. In ASP.NET Core, validation can use data annotations or FluentValidation. Invalid models usually return 400 Bad Request with validation errors.

Q175. What is appsettings.json?

appsettings.json is a configuration file used in ASP.NET Core. It stores settings such as connection strings, logging levels, JWT settings, allowed hosts, feature flags, and custom configuration. Sensitive production secrets should be stored securely, such as in environment variables, user secrets, or Azure Key Vault.

Q176. What is launchSettings.json?

launchSettings.json stores local development launch profiles. It can define application URLs, environment variables, browser launch settings, and IIS Express settings. It is used by Visual Studio, VS Code, and dotnet run during development, not as production configuration.

Q177. What is Swagger?

Swagger, commonly implemented through OpenAPI tools such as Swashbuckle, generates interactive API documentation. It shows endpoints, request/response models, status codes, and allows testing APIs from the browser. It is very useful during API development and integration.

Q178. Why API is preferred over MVC in modern apps?

APIs are preferred when the backend must serve multiple clients such as web apps, mobile apps, desktop apps, and third-party systems. API and frontend can be developed separately. MVC is still useful for server-rendered web pages, but APIs provide more flexibility for modern distributed applications.

Section 8: MVC / Razor

Q179. What is MVC architecture?

MVC stands for Model-View-Controller. Model represents data and business rules, View represents UI, and Controller handles requests, coordinates logic, and returns views or responses. MVC separates concerns and makes web applications easier to maintain.

Q180. Difference between MVC and Web API?

MVC is mainly used to return server-rendered HTML views. Web API is mainly used to return data, usually JSON, to clients such as Angular, React, mobile apps, or other services. MVC focuses on UI pages, while Web API focuses on resource endpoints and data exchange.

Q181. What is Razor engine?

Razor is the view engine used in ASP.NET Core MVC and Razor Pages. It allows mixing C# code with HTML using a clean syntax. Razor files usually have .cshtml extension and are compiled into executable code that generates HTML responses.

Q182. What is ViewBag?

ViewBag is a dynamic object used to pass data from controller to view. It is easy to use but not strongly typed, so mistakes are found at runtime. It is suitable for small temporary data, not for complex page models.

Q183. What is ViewData?

ViewData is a dictionary used to pass data from controller to view. It stores data as key-value pairs and requires type casting when reading complex objects. Like ViewBag, it is valid only for the current request.

Q184. What is TempData?

TempData is used to pass data from one request to the next request. It is commonly used after redirects, such as showing success or error messages after form submission. TempData usually uses session or cookies behind the scenes depending on configuration.

Q185. Difference between ViewBag, ViewData, and TempData?

ViewBag is dynamic and ViewData is dictionary-based; both pass data from controller to view in the same request. TempData persists data for the next request, making it useful for redirects. ViewBag and ViewData are not strongly typed, while a view model is preferred for structured data.

Q186. Where should TempData be used?

TempData should be used when data must survive a redirect, especially for one-time messages such as 'Record saved successfully' or 'Delete failed'. It should not be used for large or long-term data storage.

Q187. What is TempData.Peek()?

TempData.Peek() reads a TempData value without marking it for deletion. Normally, when TempData is read, it may be removed after the request. Peek is useful when you want to read the value but keep it available for another request.

Q188. Difference between Peek and Keep?

Peek reads a specific TempData value without removing it. Keep marks one or all TempData entries to be retained for another request after they have been read. Peek is used while reading; Keep is used to preserve already accessed TempData.

Q189. What is Partial View?

A Partial View is a reusable Razor view fragment that renders part of a page. It does not usually include the full layout. It is useful for common UI sections such as menus, cards, forms, tables, or repeated components.

Q190. Why Partial View is used?

Partial View is used to avoid duplicate UI code, improve maintainability, and break large views into smaller reusable pieces. It helps keep Razor pages clean and allows the same UI block to be reused in multiple views.

Q191. What are HTML Helpers?

HTML Helpers are methods in Razor views that generate HTML markup. Examples include `Html.TextBoxFor`, `Html.LabelFor`, `Html.EditorFor`, and `Html.ActionLink`. They make it easier to generate form fields and links based on model metadata.

Q192. Types of helpers?

Common helpers include standard HTML helpers, strongly typed HTML helpers, templated helpers, custom helpers, and Tag Helpers. Strongly typed helpers use expressions like `m => m.Name` and provide compile-time checking.

Q193. Difference between HTML Helper and Tag Helper?

HTML Helpers are C# method calls that generate HTML, such as `@Html.TextBoxFor`. Tag Helpers look like HTML attributes and elements, such as `<input asp-for="Name" />`. Tag Helpers are generally more readable and closer to normal HTML.

Q194. What are strongly typed views?

A strongly typed view is bound to a specific model type using `@model`. It provides IntelliSense, compile-time checking, and cleaner access to model properties. It is preferred over `ViewBag`/`ViewData` for structured page data.

Q195. What is Layout page?

A Layout page defines the common structure of multiple views, such as header, navbar, footer, scripts, and shared CSS. It is similar to a master page. Views render their own content inside the layout using `RenderBody` and optional sections.

Q196. What is ViewStart?

`_ViewStart.cshtml` runs before each Razor view in its folder hierarchy. It is commonly used to set the default layout for views. This avoids repeating `Layout = ...` in every view file.

Q197. What is ViewImports?

`_ViewImports.cshtml` is used to import common namespaces, tag helpers, and directives for Razor views. It helps avoid repeating `@using` and `@addTagHelper` statements in every view.

Q198. What is model in MVC?

A model represents the data and business-related structure used by an MVC application. It can be an entity model, view model, or DTO depending on the layer. In views, a model supplies the data that the UI displays or edits.

Q199. What is Razor syntax?

Razor syntax is the syntax used to combine C# and HTML in `.cshtml` files. It uses `@` to switch from HTML to C# expressions or code blocks. Example: `@Model.Name` outputs the `Name` property from the model.

Q200. Difference between TempData and Session?

`TempData` is short-lived and mainly used to pass data between two requests, especially after redirects. `Session` stores user-specific data for a longer period across multiple requests. `TempData` is suitable for flash messages; `Session` is used for broader user state but should be used carefully.

Section 9: Advanced / Scenario-Based Questions

Q201. Why generic collections are safer than non-generic collections?

Generic collections are safer because they enforce the element type at compile time. For example, `List<int>` cannot accidentally store a string. Non-generic collections store values as object, so mistakes may appear only at runtime and may require casting. Generics also avoid boxing/unboxing for value types, improving performance.

Q202. Why does LINQ use deferred execution?

LINQ uses deferred execution so queries are executed only when results are actually needed. This improves efficiency, allows query composition, and avoids unnecessary work. In EF Core, it allows the final query expression to be translated into one SQL query only when enumerated.

Q203. Why does EF Core use DbContext?

EF Core uses `DbContext` as the central unit that represents a database session. It tracks entity changes, manages relationships, executes queries, handles transactions, and saves changes. It also acts as a Unit of Work, so multiple changes can be committed together consistently.

Q204. Why are interfaces heavily used in ASP.NET Core?

Interfaces are heavily used because they support loose coupling, dependency injection, unit testing, and clean architecture. Controllers can depend on `IService` instead of a concrete service, and services can depend on `IRepository` instead of a concrete repository. This makes implementations replaceable without changing consumers.

Q205. When would you use abstract class over interface?

Use an abstract class when related classes need shared code, shared state, protected members, or a base constructor. For example, multiple report generators may share validation and logging in an abstract base class. Use an interface when you only need to define a contract and want maximum flexibility.

Q206. Why middleware order matters?

Middleware order matters because each middleware runs in the sequence it is registered. For example, `UseAuthentication` must run before `UseAuthorization` because authorization needs an authenticated user. Exception handling should be early so it can catch errors from later middleware. Wrong order can cause security, routing, or response issues.

Q207. Why use async/await in APIs?

`async/await` allows the server to handle I/O operations such as database calls, file reads, and HTTP calls without blocking threads. This improves scalability because threads are freed while waiting for I/O. In APIs, async methods are especially useful with EF Core methods like `ToListAsync` and `SaveChangesAsync`.

Q208. What happens internally when API request comes?

The request reaches the web server such as Kestrel, enters the ASP.NET Core middleware pipeline, passes through middleware like exception handling, routing, CORS, authentication, and authorization, then reaches the matched controller action. Model binding creates action parameters, validation runs, the action calls services/repositories, a result is produced, and middleware sends the HTTP response back to the client.

Q209. Explain complete request lifecycle in ASP.NET Core.

Complete lifecycle: client sends HTTP request -> Kestrel receives it -> middleware pipeline starts -> exception handling/CORS/static files/routing execute -> endpoint is selected -> authentication validates identity -> authorization checks permissions -> model binding maps request data -> model validation runs -> controller action executes -> services and repositories perform business/data logic -> result is converted to JSON or view -> response travels back through middleware -> client receives status code, headers, and body.

Q210. Explain how garbage collection works internally.

The garbage collector manages the managed heap. It starts from root references such as stack variables, static fields, CPU registers, and GC handles. It marks reachable objects, identifies unreachable objects, reclaims memory, and may compact memory to reduce fragmentation. Short-lived objects are collected frequently in Gen 0, while long-lived objects move to Gen 1 and Gen 2. This generational design improves performance.

Q211. Explain boxing/unboxing with memory.

When a value type is boxed, CLR creates an object on the heap and copies the value into it. The variable of type object stores a reference to that heap object. During unboxing, CLR checks that the object contains the expected value type, then copies the value back. This adds allocation and copy cost, so repeated boxing/unboxing should be avoided in performance-sensitive code.

Q212. Explain CLR execution flow.

CLR execution flow: source code is compiled by the language compiler into IL and metadata inside an assembly. At runtime, CLR loads the assembly, verifies type safety, resolves dependencies, and uses the JIT compiler to convert IL into native machine code. The native code executes while CLR provides memory management, exception handling, security, and garbage collection.

Q213. How does .NET support multiple languages?

.NET supports multiple languages through common IL, CTS, CLS, and CLR. C#, VB.NET, F#, and other .NET languages compile to IL. CTS ensures common type compatibility, CLS provides cross-language rules for public APIs, and CLR executes the IL. Therefore, a C# class library can be used from another .NET language.

Q214. Why is CTS important?

CTS is important because it provides a common type system for all .NET languages. Without CTS, each language might represent types differently, making cross-language integration difficult. CTS ensures that an int, string, class, interface, enum, delegate, and other types behave consistently across .NET languages.

Q215. Why is CLS needed if CTS already exists?

CTS defines all possible .NET types and rules, but not every language supports every CTS feature. CLS defines a safe subset that languages should follow for public APIs. This ensures libraries can be consumed by multiple .NET languages without compatibility issues.

Q216. Explain IEnumerable vs IQueryable with database examples.

IEnumerable works with in-memory data. If EF Core data is converted to IEnumerable too early, later filters may run in application memory. IQueryable represents a database query expression. Example: `context.Customers.Where(c => c.City == "Delhi")` as IQueryable can be translated to SQL `WHERE City = 'Delhi'`. If all customers are loaded first and then filtered as IEnumerable, more data is transferred and performance decreases.

Q217. Explain eager loading with SQL generated.

Eager loading uses `Include` to load related entities in the same query or related queries depending on EF Core strategy. Example: `context.Orders.Include(o => o.Customer)` usually produces SQL with a JOIN between Orders and Customers. This avoids separate queries when customer data is needed with orders. However, including too many collections can create large result sets, so projection to DTOs is often better.

Q218. Explain migration process internally.

When a migration is added, EF Core compares the current model snapshot with the new model. It generates a migration class containing operations like `CreateTable`, `AddColumn`, `DropColumn`, `AddForeignKey`, and `CreateIndex`. When database update runs, EF Core converts these operations into provider-specific SQL and records the applied migration in the `__EFMigrationsHistory` table.

Q219. Explain delegate with real-world use case.

A real-world delegate use case is a notification system. A payment service can accept a delegate callback to run after payment succeeds. The caller can pass different methods such as sending email, updating loyalty points, or logging. Events are also delegate-based: a button click event stores methods that should run when the button is clicked.

Q220. Explain middleware with custom middleware example.

Custom middleware is useful for cross-cutting logic such as request logging. A middleware class receives `RequestDelegate` next in its constructor and has an `Invoke/InvokeAsync` method. It can run code before and after `await next(context)`. Example logic: log request path before next, call the next middleware, then log response status code after next. This keeps logging separate from controllers.